

Experimental setup

Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (cargo, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e ".[dev,viz]"` / `pip install -e ".[all]"` as in `GETTING_STARTED.md`. Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package `README.md`.

Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("./path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("./path/to/repo", config)
```

Adjust PipelineConfig.stages, skip_stages, and plugins to

Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). `../cogant/docs/architecture/README.md` and `../cogant/docs/reference/implementation_status.md` describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```
# cogant.config.yaml
```

```
pipeline:
```

```
  stages:
```

```
    - ingest
```

```
    - static
```

```
    - normalize
```

```
    - graph
```

```
    - dynamic
```

```
# optional; skipped if no coverage/tra
```

```
    - translate
```

```
    - statespace
```

```
    - process
```

CLI

Use the cogant CLI for scripted batch runs—see `../cogant/docs/cli/README.md` for the command list that matches the installed version.

Export targets

The primary export targets are the **Generalized Notation Notation (GNN)** canonical Markdown (`model.gnn.md`) and the equivalent companion JSON files described in `../cogant/docs/export/README.md`. Optional interop targets (GraphML, Parquet) support analysis in Gephi/yEd and DuckDB, and optional tensor views for PyTorch Geometric, DGL, or HDF5 can be selected when downstream graph neural network training pipelines need to consume the program graph as a relational tensor. Ensure the Python environment includes optional dependencies for these tensor exports when those code paths are used.

Python AST parser capabilities

The v0.5.x front end relies on `cogant.static.parser.PythonASTParser`, which processes Python source through the standard-library `ast` module at the CPython version available in the runtime (3.11+ required, consistent with the `requires-python = ">=3.11"` declared in `../cogant/pyproject.toml`). The parser extracts the following construct categories:

- ▶ **Module-level entities:** module docstrings, `__all__` exports, top-level assignments.
- ▶ **Functions and methods:** `def` and `async def`, including signatures with positional, keyword, variadic (`*args`, `**kwargs`), and positional-only parameters. Default values are recorded as constant expressions where statically evaluable.
- ▶ **Classes:** class definitions, base classes, metaclasses, and the `__init__` / `__new__` boundary.
- ▶ **Decorators:** `@staticmethod`, `@classmethod`, `@property`, `@dataclass`, and arbitrary user-defined decorators. Decorator arguments are captured as attribute metadata

Progressive IR stages

Processing advances through six intermediate representations, each adding semantic detail atop its predecessor. Table 3 summarizes what each stage contributes.

Table 3. Progressive IR stages and their contributions.

Stage	IR name	Key additions	Typical output size (10K-function repo)
1	Repo IR	Raw entities and relationships per file; deduplication; merged type info	~15 MB JSON
2	Program Graph IR	Consolidated directed graph $G =$	~20 MB JSON

Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in

`../cogant/docs/architecture/README.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	< 30 s	< 500 MB
100K functions	< 5 min	< 2 GB
1M functions	< 1 hr	< 2 GB (streaming)

These are architecture targets, not benchmark claims from this manuscript. They assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.5.x release, Rust acceleration is partially wired — `cogant._rust` exposes a PyO3 `connected_components` FFI for graph construction behind the `COGANT_USE_RUST` feature flag — and a pure-Python fallback

Measured runs on packaged fixtures

The following tables record measurements taken by running the shipped RoundtripOrchestrator (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture distributed with the package. Three fixtures are the control-positive synthetic repositories under `../cogant/examples/control_positive/` (`calculator`, `event_pipeline`, `flask_mini`); the other three are real-world code under `../cogant/examples/real_world/` (`flask_app`, a six-module Flask service; `requests_lib`, a six-module reduction of the `requests` HTTP library; and `json_stdlib`, a four-module reduction of the CPython `json` package). Each run executes the full static pipeline (ingest, parse, symbols, imports, call graph, program graph, translation, state-space compilation, GNN package build, validation). Wall-clock times were measured on a single macOS workstation with the pure-Python fallback implementations — the v0.5.x PyO3 `connected_components` FFI is disabled for this canonical run (`COGANT_USE_RUST=0`) so that these numbers correspond to the Python orchestration layer with no native crates

Test matrix and coverage

The v0.5.0 Python implementation ships a test suite of **2129 passing tests** with **86 skips** for optional dependencies (Rust toolchain, matplotlib, tree-sitter language grammars, PNG rasterization) plus **2** expected xfail and **1** xpass. The suite executes in approximately four minutes on a 2024-class Apple-silicon workstation (**238** s in the canonical run recorded in METRICS.yaml), and the overall line coverage of py/cogant/ is **83.42%**, measured against executable statements reported by coverage.py on the canonical run (**2026-04-10T21:03:18.504378Z**).

Table 8. Python interpreter matrix.

Python version	pyproject.toml classifier	Status
3.11	Programming Language :: Python :: 3.11	supported (minimum version, requires-python = ">=3.11")

Mutation testing

Mutation testing was performed on the algorithmic core modules (`gnn/matrices.py`, `translate/engine.py`, `markov/blanket.py`, `statespace/compiler.py`, `static/dataflow.py`). The canonical mutmut 3.5.0 runner was evaluated but rejected: on COGANT's test layout mutmut reported every one of the 403 auto-generated mutants on `matrices.py` as “no tests” because its v3 trampoline requires tests to import the mutated module through the `mutants/<path>` shadow tree, and the project's pytest configuration does not. Rather than ship a “no tests” score, the mutation analysis in

`../cogant/docs/evaluation/MUTATION_REPORT.md` is based on a **hand-picked set of fifteen semantic mutations** that target the algorithmic predicates, constants, and loop bounds of the above modules; each mutation was applied, the relevant pytest subset was rerun, and the mutation was reverted immediately. This is a more informative experiment than a green mutmut run because it documents exactly *which* invariants the tests enforce.

Table 10. Hand-curated mutation results on COGANT

Benchmark suite (shipped)

A reproducible benchmark harness lives at `../cogant/benchmarks/bench_suite.py` and writes its canonical results to `../cogant/benchmarks/results/`. The most recent run committed to the tree (`bench(p1.5)`: Rust build status + Python vs Rust benchmark results, then superseded by `bench(suite)`: reproducible 6-fixture benchmark harness with stage timing + memory + GNN stats) executed each fixture for three iterations on CPython 3.12.11 / macOS arm64 and recorded per-stage wall-clock time, peak memory, and the final GNN tensor shapes.

Table 11. Benchmark suite results (suite_20260409.md, three iterations per fixture, CPython 3.12.11).

Fixture	Wall-clock median (ms)	Wall-clock p95 (ms)	Nodes	Edges	Mappings	Peak mem- ory (MB)
calculator	32	35	12	25	11	0.0

What to record

For reproducible experiments, record: COGANT version or commit hash, interpreter version, list of stages executed, configuration file contents (redacted), input repository commit hash, and random seeds for any learned components **outside** COGANT that consume the exports.